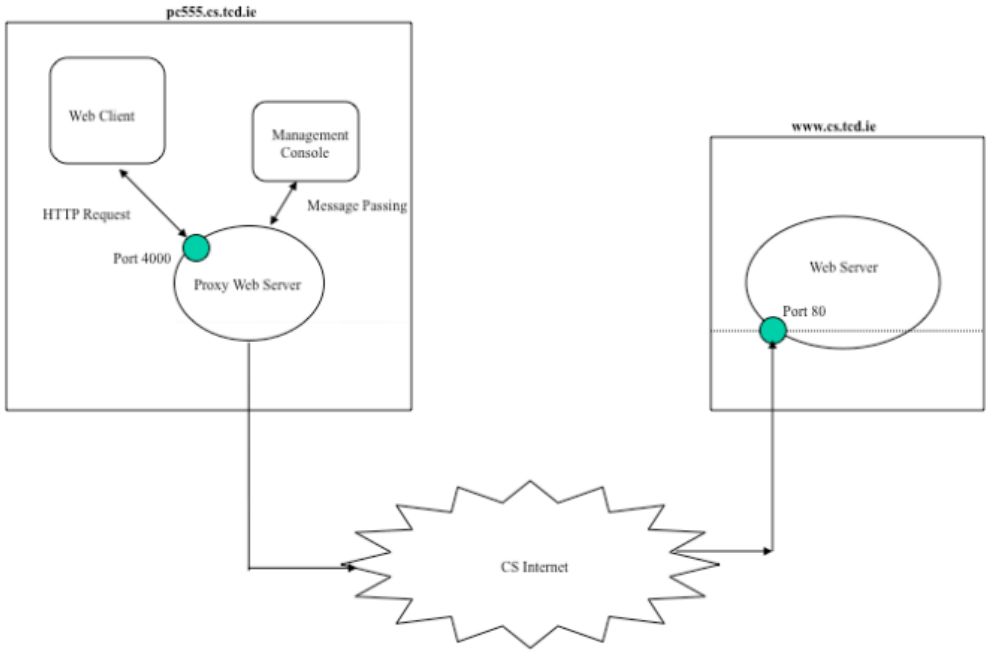
Project Documentation

Introduction

This proxy is a multi-threaded TCP server that handles both HTTP and HTTPS requests. HTTP requests are parsed and cached for efficiency, while the HTTP requests are forwarded using the `CONNECT` method. By sharing the resources (cache, blocklist, lock), the threads are able to be coordinated safely. The Management Console provides the runtime control, to allow users block/unblock hosts, inspect/clear cached responses, and view performance statistics, while the proxy handles the connections.

Requests are forwarded to the web server with responses relayed back to the browser. HTTP requests can be cached to save bandwidth with the HTTPS requests maintaining end-to-end encryption. The proxy can also dynamically block hosts & monitor cache efficiency using logs. The thread safety is maintained using thread pools and locking. These threads allow the proxy server to handle multiple client connections in the TCP Server and Management Console.



Diagram

For this project, I used Python and the import libraries `socket` for the socket programming, `threading` for parallel programming efficiency and `time` for measuring efficiency. I also utilised object-oriented programming to better modularise the program into manageable parts.

Imports, Constants & Base Class

```
import socket
import threading
import time

CACHE_TTL = 60
MAX_CACHE_SIZE = 50
MAX_CACHE_BYTES = 1_000_000
BUFFER_SIZE = 4096
SOCKET_TIMEOUT = 30
THREAD_POOL_SIZE = 20
TIMING_LOG = "timing_log.txt"

class ServerProxy:
    def __init__(self, host="127.0.0.1", port=8888, cache=None, blocklist=None, lock=None):
        self.host = host
        self.port = port
        self.cache = cache if cache is not None else {}
        self.blocklist = blocklist if blocklist is not None else set()
        self.lock = lock if lock is not None else threading.Lock()
```

The only libraries used were `socket` for socket programming, `threading` for concurrent programming, and `time` for efficiency statistics. I used OOP for this project, with a `ServerProxy` base class for each of the classes to inherit from.

`CACHE_TTL` is the time-to-live for cached responses. If a HTTP response is older than 60 seconds, they are fetched again.

The `MAX_CACHE_SIZE` allows the oldest cached entry to be removed to make space for new requests to ensure the cache does not consume too much memory.

`MAX_CACHE_BYTES` limits the cached response size individually to again ensure the cache does not consume too much memory. In this case, a HTTP response must be fetched repeatedly.

`BUFFER_SIZE` specifies the the size of each chunk to be read from sockets to allow handling large responses efficiently without overloading memory

`SOCKET_TIMEOUT` specifies the timeout for socket operations such as connecting or reading from the server to prevent the proxy from hanging infinitely.

`THREAD_POOL_SIZE` specifies the number of worker threads in the TCP server and allows concurrent handling of multiple client requests for efficiency.

`TIMING_LOG` is the file used to log cache hits, misses & request timing, for the Management Console to show cache efficiency & performance.

URL Parser

```
def parse_url(url: str):
    scheme = "http"
    if "://" in url:
        scheme, url = url.split("://", 1)
    default_port = 443 if scheme == "https" else 80
    if "/" in url:
        authority, path = url.split("/", 1)
        path = "/" + path
    else:
        authority, path = url, "/"
    if ":" in authority:
        host, port_str = authority.rsplit(":", 1)
        port = int(port_str)
    else:
        host, port = authority, default_port

    return scheme, host, port, path
```

The URL parser assumes http by default. If the URL contains `://` (think about websites like `https://google.com`), it splits into the scheme (for example, `https`) and the url (everything after the `://`)

If the scheme is `https` , we use the default port `443` , the port used for HTTPS. Otherwise, we use the default port `80` .

We also split the original website (the authority) and the page (the path) - think `http://google.com/chrome/` , where `google.com` is the authority and `/chrome` is the page. Of course, you can have nothing after the `/` which is just the default entry page.

We can also specify a port using the `:` ; for example, `example.com:8080` . Otherwise, we just use the default port.

TCP Server

```
class TCPServer(ServerProxy):
    def start_server(self):
        http_handler = Http_Handler( cache=self.cache, blocklist=self.blocklist, lock=self.lo
ck)
        https_handler = Https_Handler(cache=self.cache, blocklist=self.blocklist, lock=self.l
ock)
        self._work = []
        self._work_cond = threading.Condition()
```

```
        for _ in range(THREAD_POOL_SIZE):
            t = threading.Thread(
                target=self._worker,
                args=(http_handler, https_handler),
                daemon=True,
            )
            t.start()

        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            sock.bind((self.host, self.port))
            sock.listen(10)
            print(f"[PROXY] Listening on {self.host}:{self.port}")
            print(f"[PROXY] Set your browser proxy to 127.0.0.1:{self.port}")

            while True:
                try:
                    conn, addr = sock.accept()
                    with self._work_cond:
                        self._work.append((conn, addr))
                        self._work_cond.notify()
                except Exception as e:
                    print(f"[ERROR] Accept failed: {e}")

    def _worker(self, http_handler, https_handler):
        while True:
            with self._work_cond:
                while not self._work:
                    self._work_cond.wait()
                conn, addr = self._work.pop(0)
            self._handle_client(conn, addr, http_handler, https_handler)

    def _handle_client(self, conn, addr, http_handler, https_handler):
        try:
            raw = conn.recv(BUFFER_SIZE)
            if not raw:
                conn.close()
                return

            method, host, port, path = Request_Parser.parse(raw)

            if method == "CONNECT":
                print(f"[{time.strftime('%H:%M:%S')}] {addr[0]} — {method} {path}")
                https_handler.handle(conn, host, port)
            else:
                print(f"[{time.strftime('%H:%M:%S')}] {addr[0]} — {method} {host}{path}")
                http_handler.handle(conn, host, port, path, raw)

        except Exception as e:
            print(f"[ERROR] Client handler failed: {e}")
            conn.close()
```

Now, we have our `TCPServer` class, inheriting from the base `ServerProxy` class. It uses threading, listening for browser connections and then forwarding the `HTTP` and `HTTPS` requests using worker threads.

The `start_server` function creates HTTP and HTTPS handlers, starts worker threads, opens a TCP socket, and accepts client connections.

The `_worker()` method lets each worker wait for work in a queue, then pops a job off the queue by retrieving the connection (`conn`) and address (`addr`). It then calls the `_handle_client()` function.

The `_handle_client()` method processes one client request. It receives an initial HTTP request and parses it. If it is an HTTPS method, it uses `CONNECT` for HTTPS tunnelling. If it's a normal `HTTP`, it passes the client socket, target host and port, path, and original raw request to `http_handler`.

Request Parser

```
class Request_Parser:
    @staticmethod
    def parse(raw_request: bytes):
        CRLF = "\r\n"
        text = raw_request.decode("utf-8", errors="replace")

        try:
            start_line, header_block = text.split(CRLF, 1)
        except ValueError:
            raise ValueError("Malformed request: missing CRLF")

        parts = start_line.split(" ")
        if len(parts) != 3:
            raise ValueError("Bad request line")

        method, target, _ = parts
        port = 443 if method.upper() == "CONNECT" else 80
        host = None
        path = target

        if method.upper() == "CONNECT":
            if ":" in target:
                host, port_str = target.split(":", 1)
                port = int(port_str)
            else:
                host = target

        elif target.startswith("http://") or target.startswith("https://"):
            _, host, port, path = parse_url(target)

        else:
            path = target
            for line in header_block.split(CRLF):
                if line.lower().startswith("host:"):
                    host_header = line.split(":", 1)[1].strip()
                    if ":" in host_header:
                        host, port_str = host_header.split(":", 1)
                        port = int(port_str)
                    else:
                        host = host_header
                    break

        return method, host, port, path
```

The `Request_Parser` class contains a static method `parse` for parsing raw bytes received from the socket (`socket.recv()`). HTTP uses `\r\n` (CRLF) to separate lines, so we first decode the bytes into a string. A typical HTTP request may look like:

```
GET /index.html HTTP/1.1\r\nHost: example.com\r\nUser-Agent: ...\r\n
\r\n
```

We split the `start_line` as the first line and store everything after it as the `header_block`. If no CRLF exists, we raise an error because the request is malformed.

Next, we parse the start line into the `METHOD` (`GET` / `CONNECT`), `TARGET` (`/index.html`), and the `VERSION` (e.g., `HTTP/1.1`). The version is not used later in the code, so it is assigned to `_` and ignored. We then set the default port to `443` for HTTPS (when using `CONNECT`) or `80` for standard HTTP.

HTTP Handler

```
class Http_Handler(ServerProxy):
    def __init__(self, cache=None, blocklist=None, lock=None):
        super().__init__(cache=cache, blocklist=blocklist, lock=lock)

    def fetch_from_server(self, host, port, request_data):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(SOCKET_TIMEOUT)
            try:
                socket.inet_aton(host)
                ip = host
            except socket.error:
                ip = socket.gethostbyname(host)
            sock.connect((ip, port))
            sock.sendall(request_data)
            response = bytearray()
            try:
                while True:
                    chunk = sock.recv(BUFFER_SIZE)
                    if not chunk:
                        break
                    response.extend(chunk)
            except socket.timeout:
                pass
            return bytes(response)

    def handle(self, conn, host, port, path, raw_request):
        cache_key = f"{host}{path}"

        with self.lock:
            is_blocked = host in self.blocklist
        if is_blocked:
            conn.sendall(b"HTTP/1.1 403 Forbidden\r\nContent-Type: text/plain\r\n\r\nBlocked
by proxy.")
            conn.close()
            print(f"[BLOCKED] {host}")
            return

        with self.lock:
            cached = self.cache.get(cache_key)
        if cached:
            ts, cached_response = cached
            age = time.time() - ts
            if age < CACHE_TTL:
                t0      = time.time()
                conn.sendall(cached_response)
                elapsed = time.time() - t0
                print(f"[CACHE HIT ] {cache_key} — served in {elapsed:.4f}s (age {age:.0f}
s)")
                self._log_timing(cache_key, "HIT", elapsed)
```

```
                conn.close()
                return

        raw_request = self._set_connection_close(raw_request)
        try:
            t0       = time.time()
            response = self.fetch_from_server(host, port, raw_request)
            elapsed  = time.time() - t0
            print(f"[CACHE MISS] {cache_key} — fetched in {elapsed:.4f}s ({len(response)} byt
es)")
            self._log_timing(cache_key, "MISS", elapsed)
        except Exception as e:
            print(f"[ERROR] {host}: {e}")
            conn.sendall(b"HTTP/1.1 502 Bad Gateway\r\n\r\nProxy could not reach the serve
r.")
            conn.close()
            return

        if len(response) <= MAX_CACHE_BYTES:
            with self.lock:
                if len(self.cache) >= MAX_CACHE_SIZE:
                    oldest = min(self.cache, key=lambda k: self.cache[k][0])
                    del self.cache[oldest]
                self.cache[cache_key] = (time.time(), response)

        conn.sendall(response)
        conn.close()

    def _set_connection_close(self, raw_request):
        lines  = raw_request.split(b"\r\n")
        output = []
        found  = False
        for line in lines:
            if line.lower().startswith(b"connection:") or line.lower().startswith(b"proxy-con
nection:"):
                output.append(b"Connection: close")
                found = True
            elif line.lower().startswith(b"keep-alive:"):
                continue
            else:
                output.append(line)
        if not found:
            output.insert(1, b"Connection: close")
        return b"\r\n".join(output)

    def _log_timing(self, url, hit_or_miss, elapsed):
        with open(TIMING_LOG, "a") as f:
            f.write(f"{time.strftime('%Y-%m-%d %H:%M:%S')}, {hit_or_miss}, {elapsed:.4f}s, {u
rl}\n")
```

Now, we have our `HTTP_Handler` class. The first function, the `fetch_from_server()` function, directly contacts the remote web server. It creates a TCP socket, resolves the hostname via DNS if it does not already have an IP, connects to the web server & forwards the client's HTTP request. It reads the response in chunks until the server closes or a timeout occurs

The second function, `handle()` is the main method called by the proxy server. It creates a cache key by joining the host and the path together. It checks the blocklist and if it's blocked closes the connection. It checks the cache (`HIT`) to see if it can be used without contacting the remote server. Otherwise, it fetches from server (`MISS`) and stores in the cache with the timestamp. The oldest cached entry is removed if the cache is out of space. The response is sent to the client.

The `_set_connection_close()` rewrites the header to prevent persistent connections & make the proxy easier to manage.

The timing is logged using `_log_timing()` with timestamps, `HIT` or `MISS` , URL , and the response time.

HTTPS Handler

```
class Https_Handler(ServerProxy):
    def __init__(self, cache=None, blocklist=None, lock=None):
        super().__init__(cache=cache, blocklist=blocklist, lock=lock)

    def handle(self, conn, host, port):
        with self.lock:
            is_blocked = host in self.blocklist
        if is_blocked:
            conn.sendall(b"HTTP/1.1 403 Forbidden\r\n\r\nBlocked by proxy.")
            conn.close()
            print(f"[BLOCKED] {host}")
            return

        try:
            try:
                socket.inet_aton(host)
                ip = host
            except socket.error:
                ip = socket.gethostbyname(host)
            remote = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            remote.settimeout(SOCKET_TIMEOUT)
            remote.connect((ip, port))
        except Exception as e:
            print(f"[ERROR] HTTPS connect failed {host}:{port} — {e}")
            conn.sendall(b"HTTP/1.1 502 Bad Gateway\r\n\r\n")
            conn.close()
            return

        conn.sendall(b"HTTP/1.1 200 Connection Established\r\n\r\n")

        def forward(src, dst):
            try:
                while True:
                    data = src.recv(BUFFER_SIZE)
                    if not data:
                        break
                    dst.sendall(data)
            except Exception:
                pass
            finally:
                src.close()
                dst.close()

        threading.Thread(target=forward, args=(conn,   remote), daemon=True).start()
        threading.Thread(target=forward, args=(remote, conn),   daemon=True).start()
```

The `Https_Handler` handles the HTTPS traffic using tunnelling via `CONNECT`. It does not cache or inspect HTTP content, but creates a tunnel between client and remote server. `handle()` is called when the proxy receives a CONNECT request. The blocklist is checked and closed immediately if it is on the list. The host is resolved - it uses DNS if not already using an IP and creates a TCP connection to the actual HTTPS server. We send a `200` connection established to start the TLS handshake. Inside `handle()`, `forward()` reads from the source and sends to destination until the connection is closed. Two threads are used to do this. After setup, the proxy becomes a pipeline where:

Browser ↔ Proxy ↔ Remote HTTPS Server

The proxy does not see HTTP headers, paths or content because everything is encrypted with TLS.

Management Console

```
class Management_Console:
    def __init__(self, blocklist, cache, lock):
        self.blocklist = blocklist
        self.cache     = cache
        self.lock      = lock

    def start(self):
        threading.Thread(target=self._run, daemon=True).start()

    def _run(self):
        print("[CONSOLE] Management console ready. Type 'help' for commands.")
        while True:
            try:
                cmd = input(">> ").strip().split()
            except EOFError:
                break
            if not cmd:
                continue

            action = cmd[0].lower()

            if action == "block" and len(cmd) > 1:
                with self.lock:
                    self.blocklist.add(cmd[1])
                print(f"[CONSOLE] Blocked: {cmd[1]}")
            elif action == "unblock" and len(cmd) > 1:
                with self.lock:
                    self.blocklist.discard(cmd[1])
                print(f"[CONSOLE] Unblocked: {cmd[1]}")
            elif action == "show" and len(cmd) > 1 and cmd[1] == "blocklist":
                with self.lock:
                    bl = set(self.blocklist)
                print("\n".join(f"  {h}" for h in sorted(bl)) if bl else "  (empty)")
            elif action == "show" and len(cmd) > 1 and cmd[1] == "cache":
                with self.lock:
                    snapshot = dict(self.cache)
                if snapshot:
                    print(f"  {'URL':<55} {'SIZE':>8}  AGE")
                    for k, (ts, v) in snapshot.items():
                        print(f"  {k:<55} {len(v):>6}B  {time.time()-ts:.0f}s ago")
                else:
                    print("  (cache is empty)")

            elif action == "clear" and len(cmd) > 1 and cmd[1] == "cache":
                with self.lock:
                    self.cache.clear()
                print("[CONSOLE] Cache cleared.")
            elif action == "stats":
                self._print_stats()
            elif action == "help":
```

```
                print("Commands:")
                print("1. block <host> - block a host")
                print("2. unblock <host> - unblock a host")
                print("3. show blocklist - list blocked hosts")
                print("4. show cache - list cached responses")
                print("5. clear cache - empty the cache")
                print("6. stats - show timing statistics")
                print("7. help - show this message")
            else:
                print("  Unknown command. Type 'help'.")

    def _print_stats(self):
        """Reads timing_log.txt and prints hit/miss summary with speedup ratio."""
        try:
            hits, misses          = 0, 0
            hit_times, miss_times = [], []
            with open(TIMING_LOG, "r") as f:
                for line in f:
                    parts = line.strip().split(", ")
                    if len(parts) < 3:
                        continue
                    kind    = parts[1]
                    elapsed = float(parts[2].replace("s", ""))
                    if kind == "HIT":
                        hits += 1
                        hit_times.append(elapsed)
                    elif kind == "MISS":
                        misses += 1
                        miss_times.append(elapsed)

            total = hits + misses
            print(f"\n  CACHE STATISTICS")
            print(f"  Total requests  : {total}")
            print(f"  Cache hits : {hits}  ({100*hits//total if total else 0}%)")
            print(f"  Cache misses : {misses}  ({100*misses//total if total else 0}%)")
            if hit_times:
                print(f"Avg HIT time : {sum(hit_times)/len(hit_times):.4f}s")
            if miss_times:
                print(f"  Avg MISS time   : {sum(miss_times)/len(miss_times):.4f}s")
            if hit_times and miss_times:
                avg_hit  = sum(hit_times)  / len(hit_times)
                avg_miss = sum(miss_times) / len(miss_times)
                if avg_hit > 0:
                    ratio = avg_miss / avg_hit
                    print(f"  Cache is {ratio:.1f}x faster than origin")
                else:
                    print(f"  Cache is effectively instant (HIT time < 0.0001s)")
        except FileNotFoundError:
            print("  No timing data yet - make some HTTP requests first.")
```

The management console is a threaded console for controlling and monitoring the proxy server. You can block & unblock hosts, view & clear the cache, and inspect cache performance using it. It waits for user input. The following commands can be used:

```
block <host>
unblock <host>
```

`self.lock` is used to update the blocklist.

```
show blocklist
show cache
```

The commands above take a snapshot of the cache or blocklist. It shows the URL, response size in bytes, and time since cached.

```
clear cache
```

To clear the cache, run the above command. The threads are locked to prevent errors.

```
stats
```

`stats` reads `TIMING_LOG` from `Http_handler` . It shows the number of cache hits/misses, average hit & miss times & how much faster the cache is compared to fetching from the server.

```
help
```

`help` prints out all the possible commands. Unknown commands also show an error message.

Meanwhile, `_print_stats()` reads the timing log, separates `HIT` and `MISS` entries, computes the totals, averages, and speeds from caches.

Main

```
if __name__ == "__main__":
    shared_cache     = {}
    shared_blocklist = set()
    shared_lock      = threading.Lock()
    Management_Console(shared_blocklist, shared_cache, shared_lock).start()
    TCPServer(
        host      = "127.0.0.1",
        port      = 8888,
        cache     = shared_cache,
        blocklist = shared_blocklist,
        lock      = shared_lock,
    ).start_server()
```

Main is the entry point of our program, using a shared cache, blocklist and lock, to ensure our threads have all the necessary data, including among HTTP/HTTPS handlers and the management console. The management console is started, along with the TCP proxy server.

Efficiency

timing_log.txt

```
2026-02-28 21:00:51, MISS, 3.3629s, 34.223.124.45/
2026-02-28 21:01:08, HIT, 0.0000s, 34.223.124.45/
2026-02-28 21:39:36, MISS, 0.0557s, clients2.google.com/time/
2026-02-28 21:41:36, MISS, 3.9313s, 34.223.124.45/
2026-02-28 21:41:39, MISS, 0.0572s, www.microsoft.com/pki/cer
2026-02-28 21:41:39, MISS, 0.0971s, www.microsoft.com/pki/cer
2026-02-28 21:41:39, MISS, 0.0345s, www.microsoft.com/pki/cer
2026-02-28 21:41:39, MISS, 0.0331s, www.microsoft.com/pki/cer
2026-02-28 21:48:52, MISS, 11.4234s, 34.223.124.45/
2026-02-28 21:49:20, HIT, 0.0000s, 34.223.124.45/
```

```
── Cache Statistics ──────────────────────
Total requests  : 2
Cache hits      : 1  (50%)
Cache misses    : 1  (50%)
Avg HIT time    : 0.0000s
Avg MISS time   : 3.3629s
Cache is effectively instant (HIT time < 0.0001s)
```

Above is a screenshot from the `timing_log.txt` and a screenshot of the output of the `stats` command. As you can see, cache `HIT`s are significantly faster than `MISS`es, with cached responses being effectively instant (average time of `0.0000s`), whereas a `MISS` (requiring a fetch from the origin server) takes a couple of seconds. Since the HIT time is so small, the speedup ratio is effectively zero, so we print our default statement to prevent division by zero, indicating that the response is effectively instant, representing a 100% improvement in response time.